

Relazione Fondamenti di Elettronica

Robert Daniel Bararu 742403

Vincenzo Ingiaimo 741831

Ilaria Larovere 741467

July 2025

Contents

Obiettivo	2
ESP32	2
BME688	2
Specifiche Tecniche BME688	3
Modulo GPS NEO-6M-0-001	3
Micro SD Card Adapter	3
Schema Circuitale	4
Datasheet	5
Codice Sorgente	6

Obiettivo

L'obiettivo del progetto è la realizzazione di una stazione meteo, utilizzando come microcontrollore la **ESP32**. Oltre all'ESP32, è stato impiegato un modulo **GPS NEO-6M-0-001** per la rilevazione della posizione corrente in termini di latitudine e longitudine, un sensore **Adafruit BME688** per la misurazione della pressione atmosferica, della temperatura e dell'umidità ed un modulo **Micro SD Card Adapter** per l'archiviazione dei dati raccolti.

ESP32

Il microcontrollore utilizzato per il progetto è la **ESP32**, una scheda di sviluppo basata su un chip SoC (System on Chip). L'ESP32 offre ottime prestazioni computazionali.

Una delle principali caratteristiche della ESP32 è l'integrazione di **Wi-Fi** e **Bluetooth**, che permette di sviluppare applicazioni IoT con connettività wireless senza l'uso di moduli esterni.

Dispone di numerosi **GPIO (General Purpose Input/Output)** configurabili, di interfacce digitali.

Grazie alla sua flessibilità e al supporto per ambienti di sviluppo come **Arduino IDE**, **PlatformIO** o **ESP-IDF**, l'ESP32 è particolarmente adatta per progetti embedded, di automazione, monitoraggio ambientale e applicazioni IoT.

BME688

Per il monitoraggio ambientale della stazione meteo è stato utilizzato il modulo **Adafruit BME688**, un sensore digitale avanzato in grado di rilevare **temperatura, pressione atmosferica, umidità relativa e qualità dell'aria**.

Il modulo comunica tramite interfaccia digitale **I²C** o **SPI** ed è ottimizzato per dispositivi portatili, IoT e applicazioni smart home grazie al suo ingombro ridotto (3.0 x 3.0 x 0.93 mm) e alla possibilità di funzionare in modalità **ultra-low power**.

Specifiche Tecniche BME688

Parametro	Valore Tipico	Unità
Dimensioni	3.0 x 3.0 x 0.93	mm
Interfacce digitali	I ² C (3.4 MHz), SPI (10 MHz)	–
Tensione di alimentazione (VDD)	1.71 – 3.6	V
Temperatura operativa	-40 – +85	°C
Umidità operativa	0 – 100	% r.H.
Pressione operativa	300 – 1100	hPa
Sensore di umidità		
Accuratezza	±3	% r.H.
Sensore di pressione		
Accuratezza assoluta	±0.6	hPa
Sensore di temperatura		
Accuratezza assoluta	±0.5 (0–65°C)	°C

Table 1: Specifiche principali del sensore BME688

Modulo GPS NEO-6M-0-001

Per rilevare la posizione geografica della stazione meteo è stato impiegato il modulo **GPS NEO-6M-0-001**, noto per le sue buone prestazioni in ambienti embedded e IoT.

Il modulo è in grado di ricevere dati da un massimo di **22 satelliti** in contemporanea e supporta fino a **50 canali** di tracciamento. La precisione orizzontale tipica è di circa **2.5 metri CEP**. Il tempo di acquisizione della posizione (TTFF) è molto rapido: **1 secondo per hot start**, mentre per un cold start può impiegare circa **27 secondi**.

Il GPS NEO-6M comunica con il microcontrollore (ESP32) tramite **interfaccia UART**. Il modulo è alimentato a una tensione compresa tra **2.7V e 3.6V** e consuma circa **45mA** in modalità di tracciamento attivo.

Grazie alla sua compattezza, al basso consumo e alla semplicità d'integrazione, il modulo NEO-6M si è rivelato adatto per fornire le coordinate geografiche necessarie al funzionamento completo della stazione meteo.

Micro SD Card Adapter

Per la memorizzazione dei dati raccolti dalla stazione meteo è stato utilizzato un modulo **MicroSD Card Adapter**, un lettore/scrittore per schede di memoria compatibile con **schede MicroSD e MicroSDHC**.

Il modulo si interfaccia con il microcontrollore (ESP32) tramite **interfaccia SPI**, ed è dotato di un circuito di **conversione di livello** che consente la compatibilità con tensioni di alimentazione a **3.3V e 5V**, rendendolo adatto

sia per sistemi a bassa tensione come l'ESP32, sia per microcontrollori basati su 5V (come Arduino Uno).

La connessione avviene tramite sei pin: GND, VCC, MISO, MOSI, SCK e CS, che rispettano lo standard SPI. Il modulo include un **regolatore di tensione LDO** per fornire alimentazione stabile a 3.3V alla scheda SD.

Grazie a questo modulo è stato possibile implementare un sistema di **data-logging**, ovvero la scrittura dei dati ambientali (come temperatura, umidità e pressione) in un file di testo all'interno della memoria MicroSD, consentendo così l'archiviazione e l'analisi successiva delle misure.

Schema Circuitale

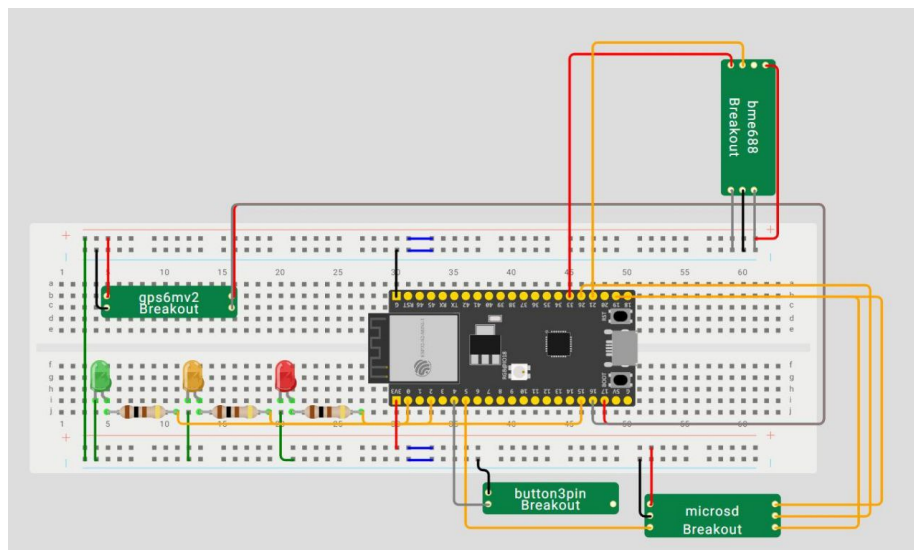


Figure 1: Schema Circuitale

Nota:

- il pin GPIO26 nello schema circuitale corrisponde al pin GPIO23 dell'ESP32 fisico (MOSI)
- il pin GPIO33 nello schema circuitale corrisponde al pin GPIO22 dell'ESP32 fisico (SCK)

Datasheet

Pin GPS	ESP32	Funzione
VCC	3.3V	Alimentazione
GND	GND	Massa
TX	GPIO16	Trasmissione dati (UART)
RX	GPIO17	Ricezione dati (UART)

Table 2: Collegamenti modulo GPS NEO-6M-0-001

Pin BME688	ESP32	Funzione
VIN	3.3V	Alimentazione
GND	GND	Massa
SCK	GPIO22	Clock I ² C (SCL)
SDI	GPIO21	Dati I ² C (SDA)
SDO	GND	I ² C
CS	GND	I ² C

Table 3: Collegamenti sensore ambientale BME688 in modalità I²C

Pin MicroSD	ESP32	Funzione
VCC	3.3V	Alimentazione
GND	GND	Massa
MISO	GPIO19	Master In Slave Out
MOSI	GPIO23	Master Out Slave In
SCK	GPIO18	Clock SPI
CS	GPIO5	Chip Select

Table 4: Collegamenti modulo MicroSD via SPI

Pin Pulsante	ESP32	Funzione
NO	GPIO4	Input digitale
C	GND	Massa

Table 5: Collegamenti pulsante a 3 pin (logica attiva alta)

Codice Sorgente

```
#include <TinyGPS++.h>
#include <HardwareSerial.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <Adafruit_Sensor.h>
#include "Adafruit_BME680.h"
#include <Wire.h>
#include <SPI.h>
#include <SD.h>
#include <FS.h>

// WiFi
const char *ssid = "Redmi";
const char *pass = "prova123";

// Timer
unsigned long ultimoInvio = 0;
const unsigned long intervallo = 30000; // invia dati ogni 30 secondi
unsigned long startTime = millis(); // tempo iniziale

// Sensori
Adafruit_BME680 bme;
TinyGPSPlus gps;
HardwareSerial gpsSerial(1); // UART1 su pin 16 (RX) e 17 (TX)

void setup() {
  Wire.begin(21, 22); // SDA, SCL
  Serial.begin(115200);
  gpsSerial.begin(9600, SERIAL_8N1, 16, 17); // RX = 16, TX = 17
  pinMode(4, INPUT_PULLUP); // Pulsante con pull-up interno
  setupLed();

  // Pulizia buffer seriali
  while (Serial.available() > 0) Serial.read();
  while (gpsSerial.available() > 0) gpsSerial.read();

  Serial.println();

  setupWifi();
  setupSensor();
  setupSD();
  readFile();

  Serial.println("Avvio lettura GPS...");
}
```

```
}

void loop() {
  // Leggo i caratteri in arrivo dal GPS

  while (gpsSerial.available() > 0) {
    char c = gpsSerial.read();
    gps.encode(c);
  }

  // Reset dati quando premi il pulsante
  if (digitalRead(4) == LOW) { // pulsante premuto (connesso a GND)
    Serial.println("Pulsante premuto, resetto dati SD...");
    resetSD();
    delay(1000);
  }

  Serial.print("Satelliti visibili: ");
  Serial.println(gps.satellites.value());

  static double ultimaTemperatura = 0; // valore persistente

  // Leggo sempre la temperatura per LED
  if (bme.performReading()) {
    ultimaTemperatura = bme.temperature;
    Serial.println(ultimaTemperatura);
    checkLed(ultimaTemperatura); // Aggiorna il LED in base alla temperatura
  }

  // Invio dati ogni 30 secondi
  if (millis() - ultimoInvio >= intervallo) {
    ultimoInvio = millis();

    // Se ci sono nuove coordinate valide
    if (bme.performReading()) {
      double lat = 0.0;
      double lon = 0.0;
      if (gps.location.isValid()) {
        lat = gps.location.lat();
        lon = gps.location.lng();
      } else {
        Serial.println("Non ci sono abbastanza satelliti disponibili, imposto latitudine e longitudine di default");
      }
    }
  }
}
```

```

double temp = bme.temperature;
double pres = bme.pressure / 100.0;
double humi = bme.humidity;

// Invio dati al server o salvo su SD
if (WiFi.status() == WL_CONNECTED) {
  HTTPClient http;
  http.begin("http://10.73.107.244:8080/api/esp-data");
  http.addHeader("Content-Type", "application/x-www-form-urlencoded");
  http.setTimeout(3000);
  String dati = "lat=" + String(lat, 6) + "&lon=" + String(lon, 6) + "&temp=" + String(temp) + "&pres=" + String(pres) + "&humid=" + String(humi);
  int code = http.POST(dati);
  // 200 se è corretto
  if (code > 0 && code == HTTP_CODE_OK) {
    String risposta = http.getString();
    Serial.println("Risposta server: " + risposta);
  } else {
    Serial.println("Errore POST o assenza di Wifi");
    salvaSD(lat, lon, temp, pres, humi);
  }
  http.end();
} else {
  salvaSD(lat, lon, temp, pres, humi);
}
} else {
  Serial.println("Fix non ancora pronto o dati BME non validi");
}
}

// salva dati su SD
void salvaSD(double lat, double lon, double temp, double pres, double humi){
  Serial.println("Wifi non presente/Errore POST, procedo con la scrittura dei dati sull'SD!");

  File fl = SD.open("/gps_data.txt", FILE_APPEND);
  if(!fl) {
    Serial.println("Errore apertura file gps_data.txt");
    return;
  }
  Serial.println("File aperto, scrittura in corso...");
  fl.println("Latitudine: " + String(lat)
    + ", Longitudine: " + String(lon)
    + ", Temperatura: " + String(temp)

```

```

    + ", Pressione: " + String(pres)
    + ", Umidità: " + String(humi)
  );
  fl.flush();
  fl.close();
  Serial.println("Scrittura su SD avvenuta con successo!");
}

// resetta dati presenti su SD
void resetSD() {
  if (SD.exists("/gps_data.txt")) {
    SD.remove("/gps_data.txt");
    Serial.println("File gps_data.txt eliminato dalla SD!");
  } else {
    Serial.println("Nessun file da eliminare.");
  }
}

File fl = SD.open("/gps_data.txt", FILE_WRITE);
if (!fl) {
  fl.close();
  Serial.println("File vuoto ricreato dopo il reset.");
} else {
  Serial.println("Errore nella creazione del file vuoto.");
}

// setup per connessione al wifi/passaggio in modalità salvataggio su SD
void setupWifi() {
  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED && millis() - startTime < 10000 ) {
    delay(500);
    Serial.print(".");
  }

  if (WiFi.status() == WL_CONNECTED){
    Serial.println();
    Serial.println("Wifi connesso!");
  } else {
    Serial.println();
    Serial.println("Wifi non disponibile. Prosegui in modalità offline.");
  }
}

// setup BME680 sensor

```

```

void setupSensor() {
  if(!bme.begin(0x76, &Wire)) {
    Serial.println("Sensore BME688 non trovata");
    while(1);
  }

  Serial.println("Sensore BME688 collegato");

  // parametri per la precisione dei dati
  bme.setTemperatureOversampling(BME680_OS_16X);
  bme.setPressureOversampling(BME680_OS_4X);
  bme.setHumidityOversampling(BME680_OS_4X);
}

// setup per SD
void setupSD() {
  if (!SD.begin(5)) {
    Serial.println("Scheda SD non trovata");
    while (1);
  }
  Serial.println("Scheda SD collegata");
}

// lettura da file
void readFile() {
  Serial.println("Avvio lettura dati vecchi su Scheda SD!");
  File fl = SD.open("/gps_data.txt", FILE_READ);

  if (!fl) {
    Serial.println("Errore apertura file gps_data.txt");
    return;
  }

  Serial.println("File aperto, contenuto:");
  while (fl.available()) {
    String riga = fl.readStringUntil('\n');
    riga.trim(); // Rimuove eventuali \r\n residui
    Serial.println(riga);
  }
  fl.close();
}

// setup iniziale dei LED

```

```

void setupLed() {
  pinMode(15, OUTPUT); // ROSSO
  pinMode(2, OUTPUT); // GIALLO
  pinMode(0, OUTPUT); // VERDE
  digitalWrite(15, LOW);
  digitalWrite(2, LOW);
  digitalWrite(0, LOW);
}

void checkLed(double temp) {
  static unsigned long lastBlink = 0;
  static bool ledState = false;
  const unsigned long blinkInterval = 500; // lampeggia ogni 500ms

  unsigned long now = millis();
  if (now - lastBlink >= blinkInterval) {
    lastBlink = now;
    ledState = !ledState;

    // ROSSO: allarme
    if (temp >= 34 || temp < -2) {
      digitalWrite(15, ledState);
    }
    // GIALLO: attenzione
    else if((temp >= -2 && temp <= 10) || (temp >= 28 && temp < 34)){
      digitalWrite(2, ledState);
    }
    // VERDE: tutto ok
    else {
      digitalWrite(0, ledState);
    }
  }
}

```